

Volume 33

Sentinel Event & Messaging Bus (SEMB)

Version: 1.0

Status: Engineering Draft

Vision

Every component inside Project Sentinel communicates through one internal messaging backbone.

No module should call another module directly unless absolutely necessary.

Instead:

Mission

Every engine listens for the events it understands.

Philosophy

Project Sentinel behaves like a living ecosystem.

When something changes...

the entire platform automatically knows.

Example

A laptop finishes a verification.

Immediately:

- Evidence Engine stores evidence.
- Knowledge Engine updates relationships.
- Trust Engine recalculates trust.
- Dashboard updates.
- Notification Engine checks policies.
- Compliance Engine updates controls.
- Digital Twin updates history.

Nobody manually triggers this.

Event Object

Every event has exactly the same structure.

Example

Event ID

Event Categories

Examples

Mission Events

Evidence Events

Workflow Events

Asset Events

Identity Events

Endpoint Events

Cloud Events

Website Events

API Events

Compliance Events

Notification Events

Audit Events

AI Events

Everything is an event.

Event Flow

Example

Website Scan Completed

No duplicated logic.

Event Timeline

Every object has
its own event timeline.

Example

Laptop

↓

Created

↓

Enrolled

↓

Baseline Created

↓

Patch Installed

↓

Trust Increased

↓

Policy Updated

↓

Mission Completed

↓

Today

Event Replay

One of my favorite ideas.

Every event
can be replayed.

Meaning

Project Sentinel can reconstruct
exactly
what happened.

Example

Yesterday

↓

Evidence Created

↓

Policy Changed

↓

Trust Dropped

↓

Recommendation Generated

↓

Verified

↓

Closed

This becomes incredibly valuable for investigations and audits.

Event Correlation

Instead of treating events independently.

We connect them.

Example

Patch Installed

The platform understands the sequence.

Event Reliability

Every event has

Status

Pending

Processing

Completed

Verified

Archived

Failed

Nothing disappears.

Event Priority

Levels

Critical

High

Medium

Low

Informational

Priority affects workflows,
not evidence quality.

Event Subscription

Each engine subscribes only to events it needs.

Example

Trust Engine

Subscribes to

Evidence Events

Policy Events

Mission Events

Configuration Events

Nothing else.

This keeps the platform efficient.

Event Persistence

Rather than relying only on a relational database, Project Sentinel maintains an append-only event history for traceability.

Events are immutable once committed.

Corrections are represented as new events that reference earlier ones.

This preserves a complete audit trail while allowing the current object state to evolve through the Object Engine.

NEW CORE ENGINE

Event Correlation Engine

This engine answers
Which events
belong together?

Example

Twenty events

↓

One Investigation

↓

One Incident

↓

One Report

NEW CORE ENGINE

Event Compression Engine

One enterprise
may generate
billions
of events.

The Compression Engine

stores historical information efficiently while preserving integrity and the ability to reconstruct timelines.

NEW CORE ENGINE

Event Integrity Engine

Every event receives

Integrity Hash

Digital Signature (optional)

Verification Status

Source Verification

Chain Position

This supports strong auditability and tamper detection.

NEW IDEA

Event Observatory

A real-time visualization

of the platform itself.

Imagine

every event

flowing through

Project Sentinel

like stars

moving through a galaxy.

Users can zoom

Organization

↓

Department

↓

Mission

↓

Object

↓

Evidence

↓

Event

This is both operationally useful and visually intuitive.

NEW IDEA

Event Time Machine

Not science fiction.

Because every object has:

- Events
- Evidence
- Missions
- Knowledge
- Trust history

we can reconstruct its state at a chosen point in time.

Example:

"Show me the Finance department as it looked on 15 March."

The platform rebuilds the known state from recorded events and evidence.

NEW IDEA

Universal Event Language (UEL)

Every internal message follows one documented standard.

Benefits:

- Easier plugin development.
- Easier testing.
- Easier debugging.
- Easier SDK development.
- Easier AI integration.
- Easier future expansion.

This becomes the internal language of Project Sentinel.

PS-ADR-0028

Event-Driven Platform

Status: Accepted

Decision

Project Sentinel shall use an event-driven architecture where major state changes are communicated through the Sentinel Event & Messaging Bus rather than direct module-to-module integration.

Rationale

An event-driven architecture improves scalability, modularity, observability, testing, and long-term maintainability while allowing new engines to be added without tightly coupling them to existing components.

NEW LONG-TERM VISION

Sentinel Cyber Nervous System

I think we should formally describe the Event Bus as:

The Nervous System of Project Sentinel.

If the Object Engine is the memory...

and the Mission Engine is the brain...

then the Event Bus is the nervous system that carries signals throughout the platform.

That gives us a consistent mental model:

- **Object Engine** → Memory
 - **Knowledge Engine** → Understanding
 - **Mission Engine** → Planning
 - **Workflow Engine** → Execution
 - **Evidence Engine** → Observation
 - **Trust Engine** → Evaluation
 - **Decision Engine** → Guidance
 - **Event Bus** → Communication
-

One more architectural improvement

I would like to introduce a principle that I believe will make Project Sentinel uniquely maintainable over the next 10–20 years:

Every engine must be independently replaceable.

For example, if in five years we develop a faster Trust Engine or a more advanced Knowledge Engine, we should be able to replace that component without rewriting the rest of the platform.

This means every engine will communicate only through documented interfaces, object contracts, and the Universal Event Language. Combined with our Python-first architecture, this gives Project Sentinel the flexibility to evolve continuously without accumulating the legacy problems that affect many long-lived enterprise platforms.

Where we are now

With Volume 33 completed, we have designed the **operating backbone** of Project Sentinel.

The next phase becomes even more exciting:

Volume 34 – Sentinel Object Engine (SOE) Internal Kernel

This will define the actual runtime kernel that manages every object, relationship, event, lifecycle, identifier, and state transition across the entire platform.

In my opinion, **this kernel will become the single most important piece of engineering in the whole project.** If we design it correctly, everything else—from website assessments and mobile security to enterprise SOC operations—will build naturally on top of it. That is the point where Project Sentinel truly becomes a Cyber Security Operating Platform rather than just a collection of security applications.

====[USER]====

Good and continue

====[ASSISTANT]====

Excellent. I think we are now about to design what will become **the heart of the entire platform.**

Everything we've designed over the last weeks has been preparing for this.

If this module is engineered correctly, Project Sentinel will be able to evolve for decades without major redesign.

This is the module that every other engine depends on.

PROJECT SENTINEL BLUEPRINT

Volume 34